

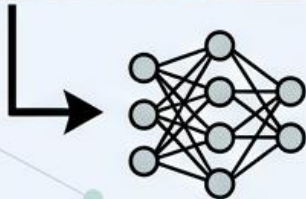
Adversarial Examples

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State
Professor, Electrical and Computer (ECE) Engineering, NC State
NAIRR AI Education Fellow, Computing Research Association (CRA)

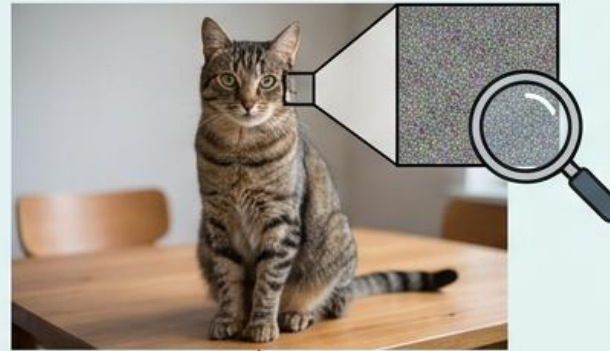
Adversarial Examples

1. CORRECT CLASSIFICATION.



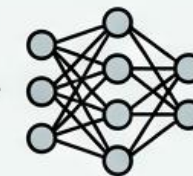
Input: Cat → Prediction: CAT
(99.8% Confidence)

2. MINIMAL PERTURBATION ADDED.



Adversarial Noise
(Imperceptible)

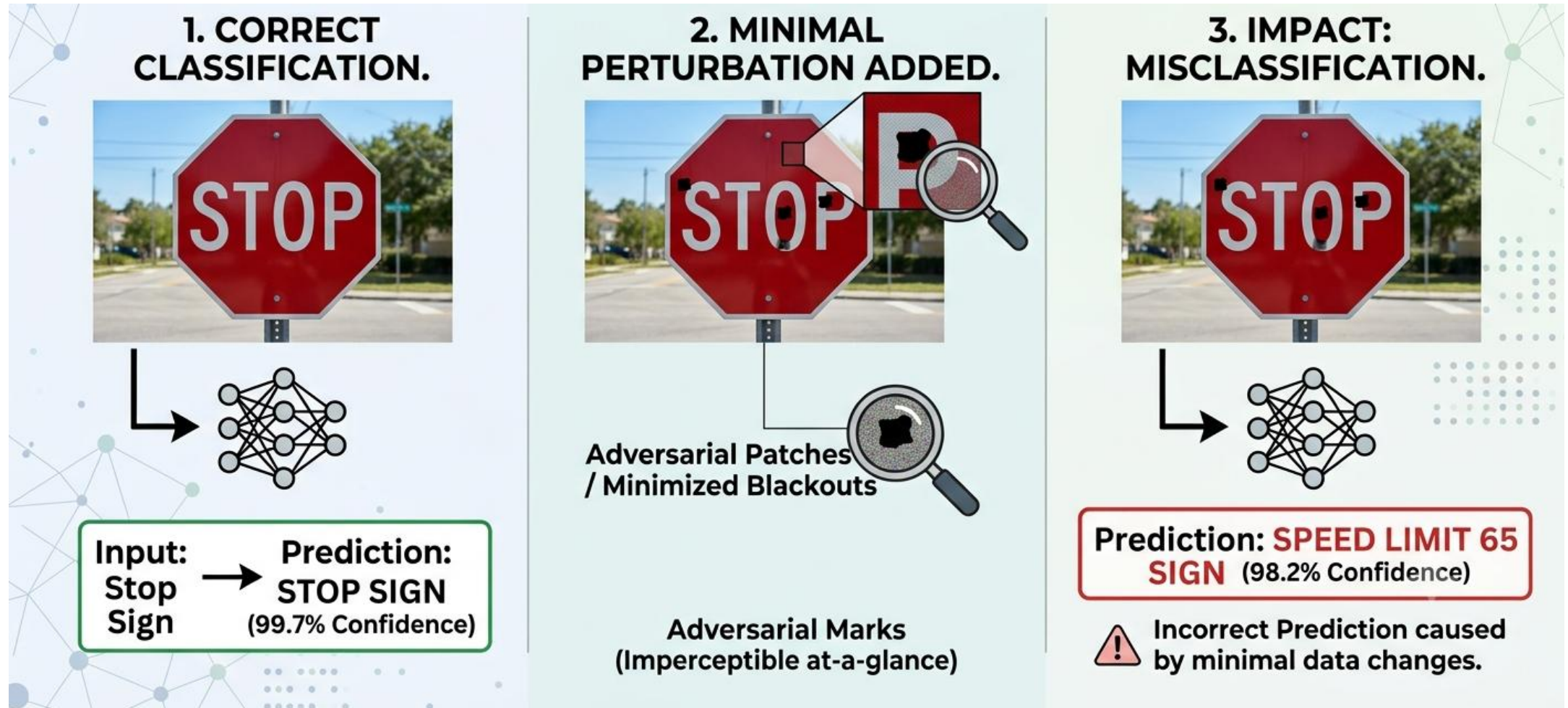
3. IMPACT: MISCLASSIFICATION.



Prediction: **TOASTER**
(98.5% Confidence)

⚠️ Incorrect Prediction caused by minimal data changes.

Adversarial Examples



Differential Approach

- Small perturbations on the right directions can lead to large error rates (~80%)


 x

“Panda”
57.7% Confidence

 $+ .007 \times$

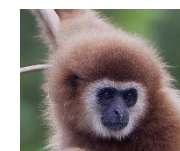
 $sign(\nabla_x C(\theta, x, y))$

“Nematode”
8.2% Confidence


 $=$

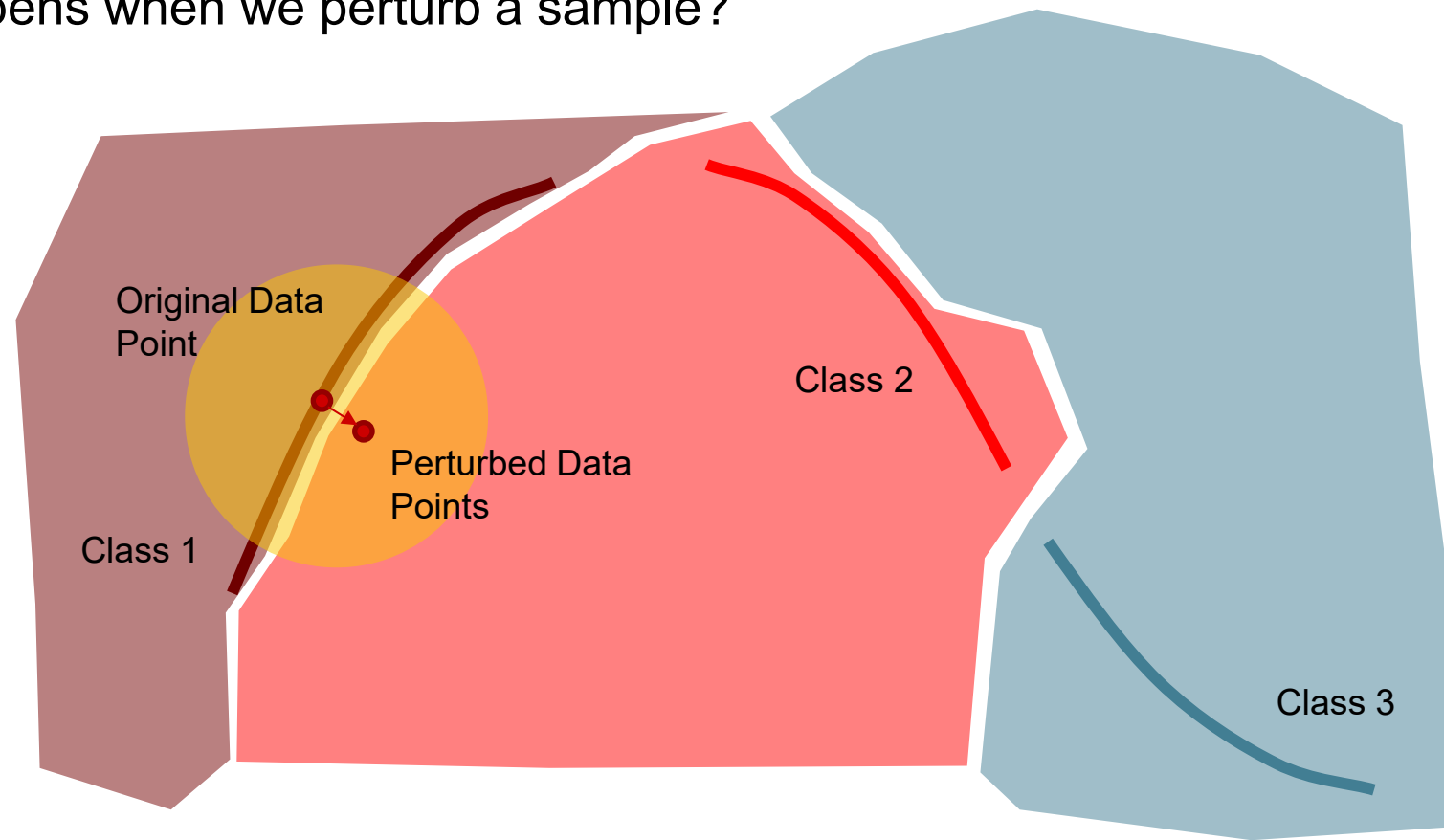
 $x + \epsilon \cdot sign(\nabla_x C(\theta, x, y))$

“Gibbon”
99.3% Confidence



Differential Approach

- What happens when we perturb a sample?



Why Do We Care?

Digital attacks (e.g., pixel perturbations) and physical attacks (small corruptions to signs)

White-Box

The attacker has full access to the model's architecture and gradients — as in FGSM.

Black-Box

The attacker only sees inputs and outputs — no internal access, often via a substitute model.

Targeted

The attack forces one specific wrong label — e.g., “classify this as a gibbon.”

Untargeted

The attack just needs any wrong label — easier to pull off, less controlled.

Countermeasures

Adversarial Training

- Bake robustness directly into the model: generate adversarial examples during training and include them in the training set, so the model learns to classify them correctly too.
- Modified loss:

$$\alpha \cdot L(y, f_{\theta}(x)) + (1 - \alpha) \cdot L\left(y, f_{\theta}\left(x + \epsilon \cdot \text{sign}(\nabla_x C(\theta, x, y))\right)\right)$$

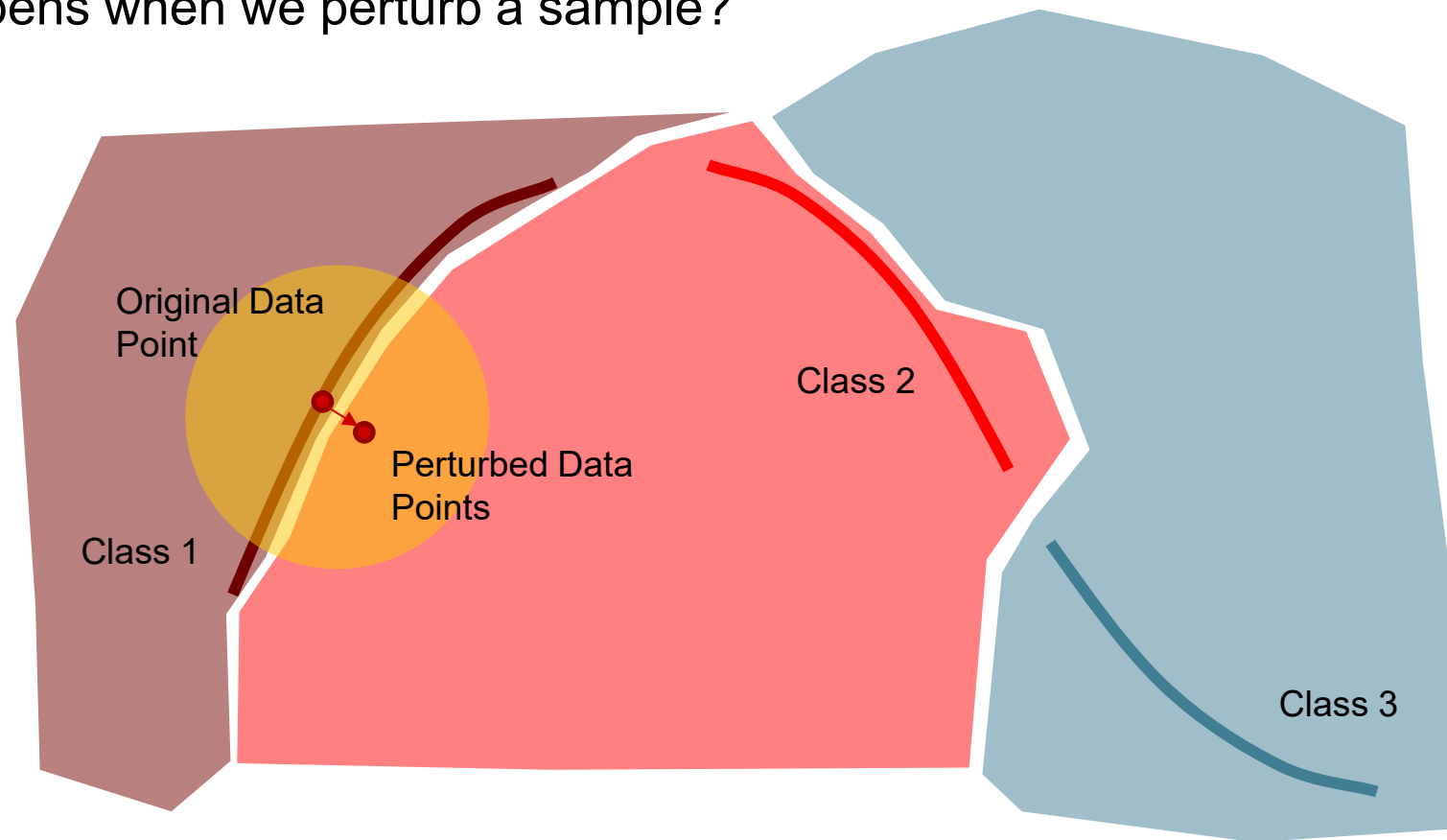
- **Trade-Off:** It may drop some performance

Purification

- Leave the classifier untouched. Instead, run the input through a separate generative model first, to “clean” the perturbation away before classification.
- **Advantage:** It works as a plug-in defense, without retraining the classifier

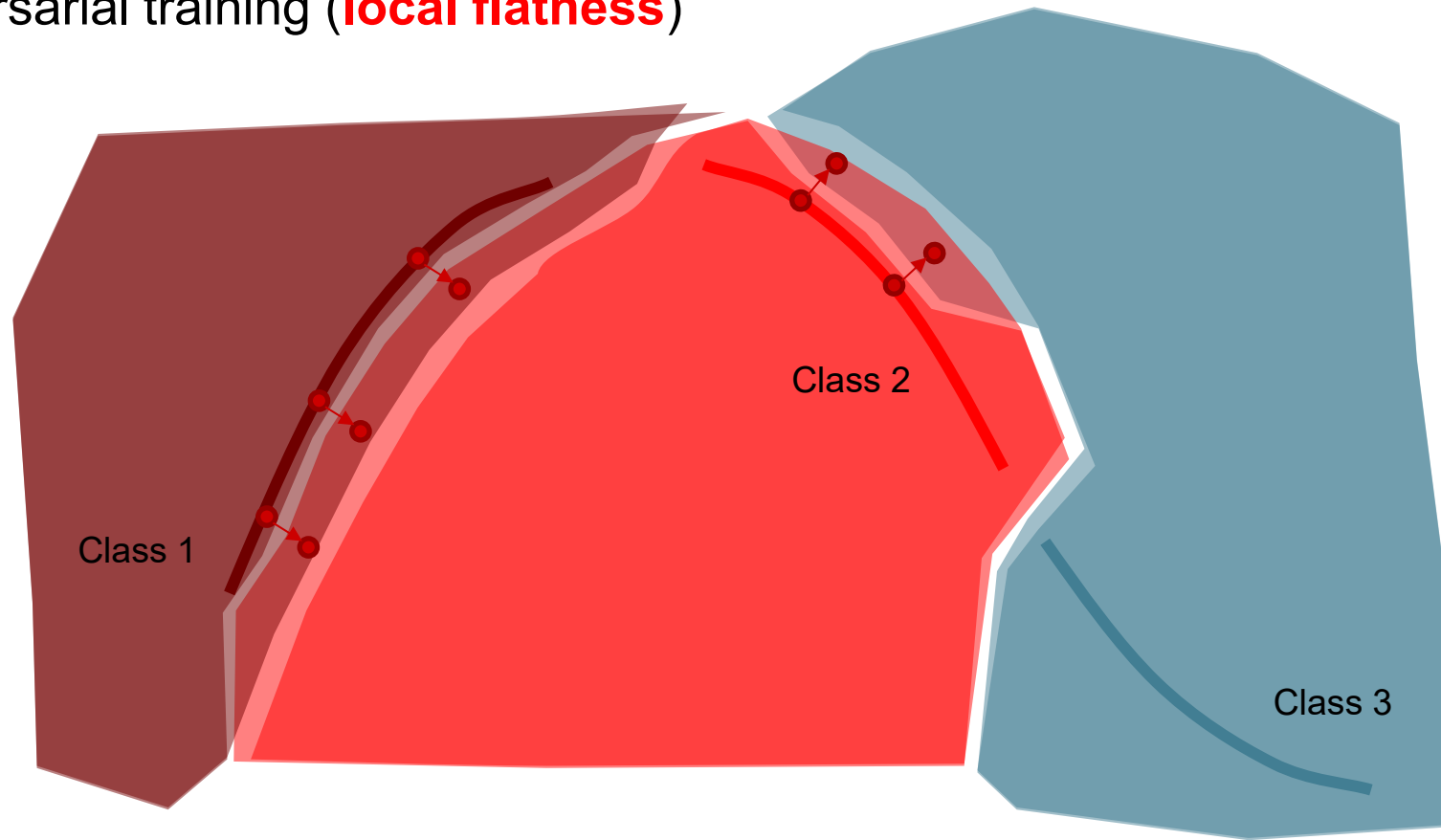
Differential Approach

- What happens when we perturb a sample?



Adversarial Training

- After adversarial training (**local flatness**)



Generative AI Has Adversarial Examples Too

- **LLMs have their own version:** adversarial prompts, or “jailbreaks” — carefully crafted text designed to bypass safety training
- Same idea as a pixel perturbation, just in token space instead of pixel space — a small, engineered change that flips the model’s behavior
- Often discovered automatically, using gradient-based search over discrete tokens — conceptually the same search that FGSM performs over pixels
- **Defenses mirror the image world:** adversarial training on known attack examples, plus inference-time filtering and detection



Uncertainty Quantification

Edgar Lobaton, Ph.D.

Director of Applied AI, College of Engineering (COE), NC State
Professor, Electrical and Computer (ECE) Engineering, NC State
NAIRR AI Education Fellow, Computing Research Association (CRA)

Why Uncertainty Matters?

- A model's point prediction alone doesn't say how much to trust it — a well-calibrated '97% confident' and a wildly wrong guess can look identical on the surface
- In safety-critical engineering — structural health monitoring, autonomous vehicles, medical devices — knowing when NOT to trust a prediction matters

Point Prediction Alone

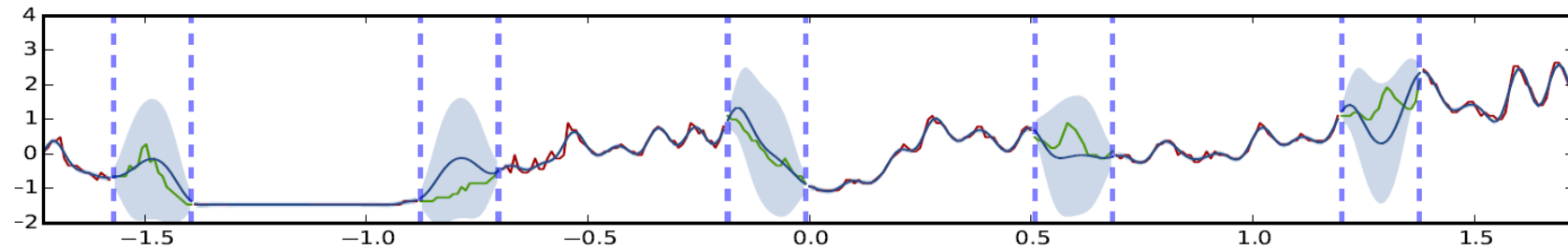
"Remaining fatigue life: 4.2 years." No indication of whether that number is solid or a coin flip.

Prediction + Uncertainty

"Remaining fatigue life: 4.2 ± 0.3 years (high confidence)" vs. " 4.2 ± 3.5 years (low confidence)" — very different decisions follow.

What Does Uncertainty Look like?

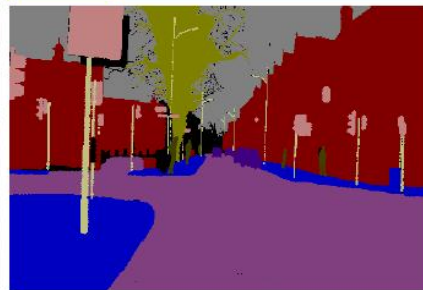
- Time Series Prediction:



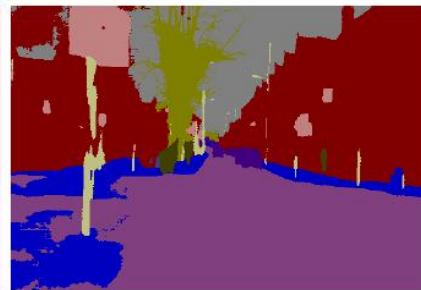
- Visual Inference:



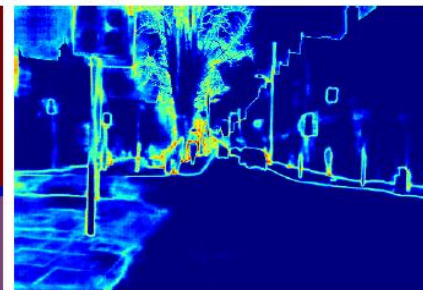
Image



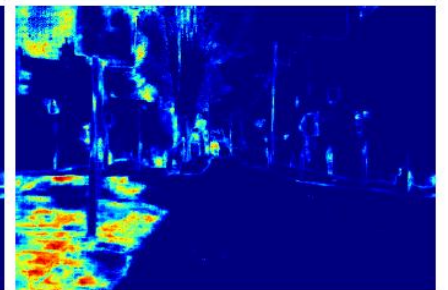
Groundtruth



Prediction



Data Uncertainty



Model Uncertainty

Uncertainty Isn't Reliable by Default

- A softmax output looks like a probability distribution — but it isn't a trustworthy one.
- Modern, high-capacity networks tend to be systematically overconfident. Ironically, more accurate architectures are often worse calibrated than older, simpler ones
- Models can output confident predictions on inputs completely unlike anything in training (Out-Of-Distribution) — they don't reliably know what they don't know

Two Types of Uncertainty

Aleatoric

Inherent in the data itself — sensor noise, natural variability. More data does NOT reduce it. “No” to the infinite-data test.

Epistemic

From the model’s limited knowledge — not enough training data in this region. More (relevant) data CAN reduce it. “Yes” to the infinite-data test.

- The distinguishing test: if you collected infinite data, would this uncertainty go away?

Do you Remember Dropout?

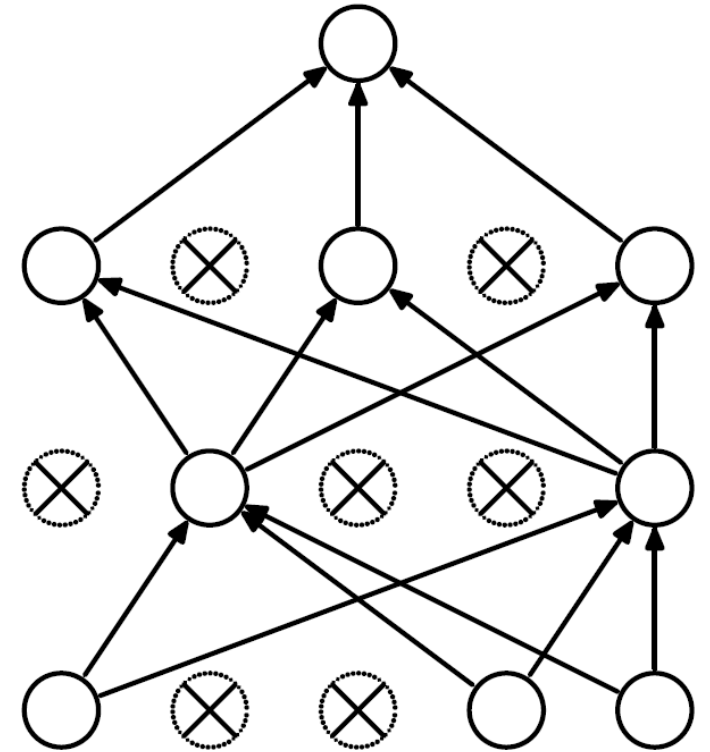
- For training using dropout, we define $r_j^{(l)} \sim \text{Bernoulli}(p)$, where $1 - p_l \in [0,1]$ is the dropout rate and j indexes the entry in the vector variable. Then,

$$\hat{W}^{(l)} = W^{(l)} \cdot \text{diag}(r^{(l)})$$

$$a^{(l)} = \hat{W}^{(l)} \cdot h^{(l-1)} + b^{(l)}$$

$$h^{(l)} = g^{(l)}(a^{(l)})$$

- We are randomly sampling from the columns of $W^{(l)}$



Estimating Uncertainty in Practice

- True Bayesian inference over a network's weights is intractable at deep-learning scale
 - these two methods are practical approximations

Monte Carlo Dropout

Keep dropout active at inference time and run many forward passes on the same input. The spread across predictions approximates epistemic uncertainty. Cheap — reuses a model you already trained.

Deep Ensembles

Train several independent models from different random initializations. Disagreement between their predictions estimates epistemic uncertainty. Often more reliable than MC Dropout, at the cost of training multiple models.

Monte Carlo Dropout

- In the case of regression with regularization, we often use:

$$C(\theta) = \frac{1}{N} \sum_i ||y_i - f_{\theta}(x_i)||^2 + \sum_l \left(\lambda_{w,l} \cdot ||W_l||_2^2 + \lambda_{b,l} \cdot ||b_l||_2^2 \right)$$

- For MC Dropout, we assume priors: $p(W) = N(W; 0, \sigma_w^2 I)$ and $p(b) = N(b; 0, \sigma_b^2 I)$ and after arrive to

$$C_{MC}(\theta) = \frac{1}{N} \sum_i \frac{||y_i - f_{\theta}(x_i)||^2}{\sigma(x_i)^2} + \frac{1}{N} \sum_i \log \sigma(x_i)^2 + \sum_l \left(\frac{p_l}{N\sigma_w^2} \cdot ||W_l||_2^2 + \frac{1}{N\sigma_b^2} \cdot ||b_l||_2^2 \right)$$

Monte Carlo Dropout

- We can now sample from the approximate posterior distribution θ to get a prediction estimate

$$E_q[\hat{y} | x] \approx \frac{1}{K} \sum_k f_{\theta_k}(x)$$

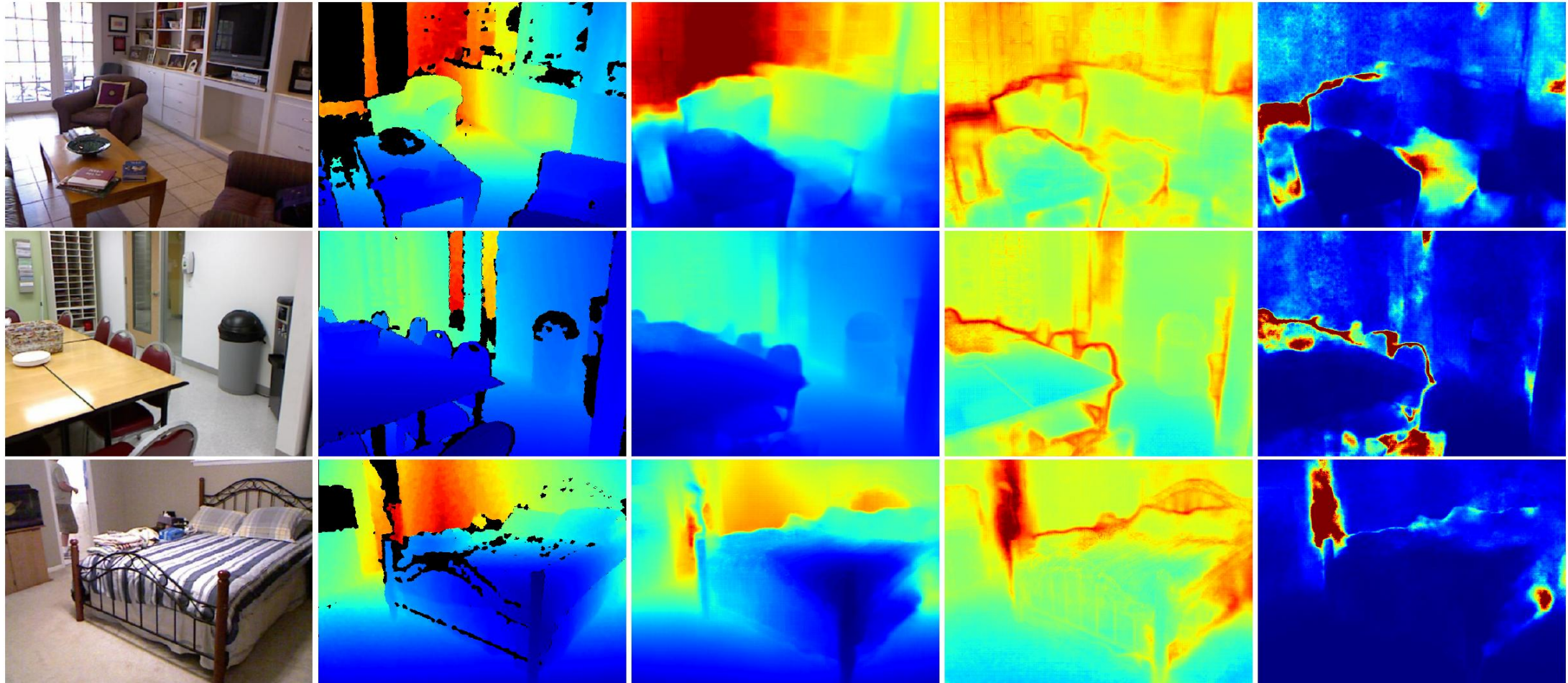
- Furthermore,

$$\text{Var}_q[\hat{y} | x] = \underbrace{\sigma(x)^2}_{\text{Aleatoric Uncertainty Term}} + \underbrace{\frac{1}{K} \sum_k f_{\theta_k}(x)^2 - E_q[\hat{y} | x]^2}_{\text{Epistemic Uncertainty Term}}$$

Aleatoric Uncertainty Term

Epistemic Uncertainty Term

Applications to Computer Vision



Image

Groundtruth

Prediction

Data Uncertainty

Model Uncertainty

Applications to Computer Vision

